Aanya Solutions — Experience Explanation

Resume verification, the corrected resume text, and interview drill-down prep for every claim.
Prepared 21 Sep 2026 from the actual source code in `backend/` and `cfc/`, plus the project docs.

> **How to use this document**
>
> 1. **Section 1**: what I checked in the code and what was wrong or overstated in the current resume.
> 2. **Section 2**: the corrected resume text, ready to paste. It covers all 5 of Priyanshu's comments.
> 3. **Sections 3–8**: one chapter per resume bullet. Each has a 30-second pitch, how it works, and likely interviewer questions with answers.
> 4. **Section 9**: concept primer (locks, deadlocks, compare-and-set, idempotency, keyset pagination, backoff).
> 5. **Section 10**: incident stories in STAR format for behavioural rounds.
> 6. **Appendix**: read-only SQL to re-verify every number before you send the resume.
>
> Rule for the whole resume: **every number and every technical term on it must be something you can explain for 5 minutes and prove from the code.** If you can't, round it down or remove it.

1. Verification: what the code actually shows

I checked each Aanya claim against the source code. I could **not** query the production database: that access was blocked in this session. So the business numbers (tickets, $, queue operations, DispatchTrack stops) are **not verified**. Run the queries in the Appendix before sending the resume.

Resume claim	Verdict	What the code / docs show	Action
"used across 3 stores"	**Partly**	The platform serves **2 physical showrooms (Arden, NC and Waynesville, NC) plus a "Web Sales" channel**, which the code treats as a third store code. The project's own security review calls the client "a two-store retailer".	Say "2 showrooms + a web-sales channel", or "3 sales locations" only if you can defend it.
3,164 tickets, $8.41M written sales	**Not verified**	Production DB access was blocked. The numbers are plausible given the analytics query (`committed=1 AND transaction_mode='SALE' AND invoice_type IN ('RG','PH')`).	Run SQL A1/A2. Put a rounded-down figure on the resume ("3,000+", "$8M+").
"built 315 REST endpoints"	**Remove**	Today there are about 305–321 route definitions across 42 route files, so the number is already stale. Priyanshu is right: it invites "name 300 endpoints" and measures effort, not impact.	Removed. Replaced with outcome (what the system replaced and protects).
"concurrency-safe floor-rotation engine … advisory locks, SELECT … FOR UPDATE, bounded deadlock retries"	**Partly**	The code is real: `upsFairness.js` has `GET_LOCK('ups_queue_lock:<store>', 5)`, FOR UPDATE row locks, and up to 2 retries on deadlock/lock-wait. **But production runs the V1 rotation** (`UPS_FAIRNESS_V2` is unset, so mode is "off"). V1's take/finish/queue-insert paths are **not** transactional. In production, the lock + transaction wrapper protects the **Be Back, Abandoned UP, Self Be Back and reward** flows. The full V2 fairness engine is dark.	Reword: "built the concurrency layer…", scoped to the flows that really use it. Never say the live rotation is fully locked.

"21,856 queue operations over 132 trading days"	**Not verified**	No numbers in the code. Even if the count is right, those operations mostly went through V1, *not* the locked engine. Attaching the number to the engine is misleading.	Only use it after SQL A3, and phrase it as board activity ("20K+ queue events"), not engine throughput.
"deployed through a shadow-rollout mechanism"	**Overstated**	There is a tri-state flag `off | shadow | on`. "Shadow" = V1 stays the truth and **dual-writes** to `ups_sessions`. There is **no V1-vs-V2 comparison/diff logging**, and shadow is **not enabled in production**.	Say "dark-launched behind an off/shadow/on flag with dual-write".
"retry-safe sale capture … single-use authorization tokens and compare-and-set"	**Verified**	Token consumed by `UPDATE … SET used=1 WHERE token=? AND user_id=? AND store_name=? AND used=0 AND expires_at>NOW()` + `affectedRows===1`. Commit uses `WHERE slug=? AND committed=0`. Drafts are guarded by `form_revision`. `SALE_TOKEN_ENABLED=true`.	Keep. Strongest bullet.
"draft → print/e-signature → immutable invoice"	**Overstated**	The Sale Lock (freeze on print) is **hard-disabled in code** (`saleLock.service.js:43`, product hold 2026-07-14). The server doesn't require a print before save; print only stamps `live_at`. Committed sales can still be deleted via an approval workflow or back-dated by an admin. What *is* true: **a committed sale can't be overwritten by a draft save or a second commit.**	Say "completed sales can't be duplicated or overwritten". Drop "immutable".
"13 external integrations"	**Overstated**	The docs list 13, but Twilio is in `package.json` and never imported, and Puppeteer/Tesseract are local libraries. **About 11 real external services**: Google Drive, Gmail SMTP, Google Sign-In (OAuth+PKCE), USPS Addresses v3, DispatchTrack XML API, RETAILvantage REST, RETAILvantage SQL Server, 2 legacy store SQL Server DBs, Birdeye reviews, Topaz SigWeb (browser).	Say "10+ third-party integrations", and be ready to list them.
"26 kill-switch flags"	**Stale**	Now **27 backend `*_ENABLED` flags + 16 frontend `VITE_*_ENABLED`**. Most are feature gates (dark launches), not integration kill switches.	Drop the exact number. Say "shipped every risky feature dark behind env feature flags".
"automated SQL → Chromium PDF → Drive → role-based email reporting"	**Partly**	The pipeline exists: node-cron 7 PM/9 PM ET, SQL aggregation, Puppeteer PDF, Drive upload, email, 3 attempts, idempotent by date, audit tables. **But recipients = admins + an env list (not role-based)**, and the **scheduled send has been paused since 2026-06-17** (SMTP port 587 ECONNREFUSED + missing Drive folder env). The manual admin trigger still works.	Drop "role-based". Know the pause story (it's actually a good interview story).
"synchronized 4,933 DispatchTrack stops"	**Word + number**	The sync is **one-way polling** of the DispatchTrack XML export into `dispatchtrack_orders` (idempotent upsert). "Stop" is only a derived grouping key. Nothing is written back to DispatchTrack.	Say "orders" / "delivery records". Verify the count with SQL A4.

"resumable SQL Server → MySQL migration pipeline, keyset pagination, checkpoints, exponential backoff"	**Verified (code)**	`services/legacyMigration/` (~1,600 LOC, 14 files). `WHERE [Id] > @afterId ORDER BY [Id]`, checkpoint in `migration_runs.last_source_id`, `withRetry` 500 ms × 2^n capped at 15 s + jitter, idempotent upsert on `UNIQUE(source_store, source_id)`. **Caveat:** it's an operator-run one-time backfill, and nothing in the repo proves it ran in production or that anything reads its output.	Say "built" (true). Don't claim "migrated N rows" unless you ran it and know N.
"production analytics *bottleneck* … correlated JSON predicate → *indexed* scalar predicate"	**Wrong framing**	It was not a performance issue. The `JSON_TABLE`-inside-`EXISTS` predicate **threw an error on the production MySQL build, so every store-filtered query returned HTTP 500**. The UI silently kept the "All Stores" data, so every store showed identical numbers. There is **no index** on the scalar; it's a per-row `JSON_EXTRACT` pruned by the `created_at` index.	Reword as a correctness/availability RCA. "Bottleneck" and "indexed" would collapse under one follow-up question.

> **Why this matters:** an interviewer who hears "concurrency-safe engine" will ask "is it in production, and how did you prove it's safe?" Today the honest answer is "V2 is dark; production uses V1 plus locked sub-flows". It is far better to write that honestly than to be caught. Every corrected bullet below is something you can defend line by line.

2. Corrected resume text (paste-ready)

2.1 Mapping Priyanshu's feedback → change

Priyanshu's comment	What I changed
"Select … for update (line) in experience section??"	No raw SQL syntax on the resume. Rotation bullet now says "row-level locking inside transactions". The syntax stays in your head for the interview (Section 4).
"Skill section keywords are less, more is better (ATS)"	Skills expanded from 4 lines to 6 grouped lines, with ~45 keywords. **Every keyword is backed by this codebase or your projects.** Items you must personally confirm are marked in the note.
"Build a resumable … (Generic)"	Rewritten to say *why* it exists (preserve legacy store data from retired systems) and *what makes it safe* (resume from checkpoint, idempotent re-runs, count reconciliation). Split from the analytics RCA, which is now its own impact bullet.
"Converted ambiguous business requirements … (Generic) replace with something quantifiable"	Replaced with a template you must fill with **real** Agility facts (I have no Agility code, so I will not invent numbers).
"315 REST endpoint … becomes flash point … just quantify impact and result"	Removed. First bullet now states what the platform replaced, who uses it, and the business volume on it.

2.2 The resume

PROFESSIONAL SUMMARY

Software Engineer with 1+ years of experience building and running production backend systems and web applications with Java, JavaScript, Node.js, React, MySQL, SQL Server and PostgreSQL. Sole engineer on a live retail platform: transactional sale capture, concurrency control, background jobs, third-party integrations and production incident response (RCA). Strong foundation in DSA (LeetCode Knight, 2045), DBMS, Operating Systems, Computer Networks and System Design.

PROFESSIONAL EXPERIENCE

Software Engineer, Aanya Solutions — Dec 2025 – Present
US-based furniture retail client · sole development engineer (design, build, deploy, production support); every change reviewed by two engineers

- Built and run the client's in-store operations platform (React, Node.js/Express, MySQL, SQL Server). It replaced paper tickets and legacy tools for sales ticketing, e-signature, returns, salesperson rotation, analytics and delivery reporting across 2 showrooms and a web-sales channel, and has captured 3,000+ sales tickets worth $8M+ in written sales.
- Built the concurrency layer for the walk-in customer rotation queue shared by many sales-floor devices: per-store MySQL advisory locks, row-level locking inside transactions and bounded deadlock retries keep hand-off, abandoned-customer and reward flows consistent; dark-launched a rewritten fairness engine behind an off/shadow/on flag with dual-write.
- Made sale capture retry-safe: single-use sale tokens consumed atomically, and compare-and-set commits, ensure a double-click, network retry or second tab can never duplicate or overwrite a completed sale; led RCAs on token-expiry and draft-overwrite incidents and shipped the fixes.
- Integrated 10+ third-party services (Google Drive/Gmail/Sign-In, USPS, DispatchTrack, RETAILvantage, Birdeye). Built a scheduled SQL → headless-Chrome PDF → Google Drive → email report pipeline with retries, idempotent runs and audit logs, and an idempotent DispatchTrack order sync with exponential backoff and a 2-year backfill.
- Diagnosed production 500s that made every store show identical Sales Analytics numbers: a JSON_TABLE predicate the production MySQL build rejected. Replaced it with a validated scalar JSON predicate across ~13 endpoints, with

regression tests and a read-only production check.

- Built a resumable SQL Server → MySQL backfill for two retired store databases: keyset pagination, DB-persisted checkpoints, exponential backoff with jitter, idempotent upserts and source-vs-target count reconciliation.

Full Stack Developer, Agilis Communication Services — Apr 2025 – Nov 2025 (keep; see note 3)
Full Stack Developer Intern, Agility Solutions LLC — Dec 2024 – Mar 2025

- Optimized SQL queries and indexing, cutting analytics dashboard load time by ~35%.
- [FILL WITH REAL FACT] Built <feature> for <users/team>, reducing <manual step / time / errors> from <X> to <Y>.

Full Stack Developer Intern, Sponsogram — Jun 2024 – Sep 2024 (unchanged)

SKILLS

Languages: Java, JavaScript (ES6+), SQL, T-SQL, HTML, CSS
Backend: Node.js, Express.js, REST API Design, Transactions & Concurrency Control, Idempotency, Background Jobs (cron), Retries & Exponential Backoff, Feature Flags, OAuth 2.0 / PKCE, Puppeteer, Kafka*
Frontend: React, Redux Toolkit, React Router, Next.js, Tailwind CSS, Vite
Databases: MySQL, SQL Server, PostgreSQL, MongoDB, Prisma, Knex, Query Optimization, Indexing, Schema Design, JSON in SQL
Testing & Tools: Vitest, Node.js test runner, Git, GitHub, Postman, Google Cloud APIs (Drive, Gmail), XML/REST third-party integrations
Computer Science: Data Structures & Algorithms, DBMS, Operating Systems, Computer Networks, OOP, System Design (HLD/LLD)

2.3 Notes you must act on before sending

1. **Highlighted numbers**: run Appendix SQL A1/A2. If the real count is ≥ 3,000 and ≥ $8M, keep them. Rounding *down* is safer than exact figures: nobody asks you to defend "3,000+", but everyone asks about "3,164".
2. **"Kafka*" and "PostgreSQL / MongoDB / Prisma"**: none of these are in the Aanya code. Keep them only if hireSight/QuickCart or earlier jobs really used them and you can answer basics (Kafka: topic, partition, consumer group, offset, at-least-once). Otherwise delete Kafka. A keyword you can't defend is worse than a missing one.
3. **Agilis bullets** are equally generic (Priyanshu didn't flag them, but a second reviewer will). Replace at least one with a specific: "Built X used by N users/day", "Cut API error rate from X% to Y%", "Resolved N production incidents".
4. **Java**: your Aanya work is JavaScript. Java-first roles (like GS) will test Java in DSA/LLD. That's fine: your 1000+ problems are in Java. Just be ready to say "production work in Node.js; DSA and LLD in Java".
5. **One page**: if it spills, drop the 6th Aanya bullet (migration) first. It has the least proven production impact.
6. **LinkedIn outreach**: you said "Top 1.96%" in one message and "Top 2%" on the resume. Use one ("Top 2%") everywhere.

3. Platform overview (bullet 1) — know the whole system

> **30-second pitch:** "I'm the only developer for a US furniture retailer with two showrooms and a web-sales channel. I built their in-store platform end to end: React front end, Node/Express API, MySQL for the app data, and read access to their SQL Server ERP (RETAILvantage). Salespeople use it all day. It runs the queue that decides who gets the next walk-in customer, the sales ticket and e-signature, returns, and the manager analytics and reports. It has captured over three thousand sales tickets, about $8M in written sales. Two senior engineers review everything I ship."

Architecture you should be able to draw on a whiteboard

```
Browser (React 19 + Redux Toolkit + Vite, Tailwind)  -- sales floor PCs / tablets
     |  axios, JSON, header x-user-id / X-CFC-Env
     v
Node.js + Express API  (single process; ~42 route files, services/, jobs/)
  |-- MySQL  cfc_form  : app data (invoices_archive, ups_* queue tables, users, audit tables)
  |-- MySQL  db_cfc    : reporting/inventory mirror
  |-- SQL Server (RETAILvantage ERP): read-only sale / item / customer lookups
  |-- node-cron jobs   : daily reset 9 PM, daily report, DispatchTrack sync, suspended-ticket GC
  |-- Puppeteer (headless Chromium): invoice + report PDFs
  '-- Google Drive / Gmail SMTP / USPS / DispatchTrack / Birdeye / Google Sign-In
```

Modules (what the salespeople / managers actually use)

- **UPS board**: rotation queue of salespeople for walk-in customers ("UPs"). Actions: join, take UP, finish, break/return, Be Back (hand a returning customer to their original seller), Abandoned UP, idle lock on shared terminals.
- **Sales form**: build the ticket (items, packages, delivery, tax, payments), print, Topaz e-signature, save. Also suspended tickets, quotes (QS) → sale conversion, live tickets.
- **RTS (returned to stock)**: return / exchange with its own calculation engine (V2 fixed a tax asymmetry) and a return ledger that blocks over-returns (409).
- **Analytics & reports**: Sales, Vendor, SB analytics, daily salesperson summary PDF, executive email, attendance exceptions, delivery analytics from DispatchTrack.
- **Admin**: user management (roles admin / manager / employee, deactivation with forced logout), demo mode with separate `demo_` tables for training.

Drill-down questions

Q. Why MySQL and not Postgres/Mongo?

The client already hosted MySQL next to their SQL Server ERP, and the data is relational (tickets, users, queue rows, audit logs). The one document-like piece, the ticket body, is stored as a JSON column (`form_json`). That gave schema flexibility for a form that changed weekly, while ids, status flags and timestamps stay in real indexed columns.

Q. Storing the whole ticket as JSON — what did that cost you?

Analytics must use `JSON_EXTRACT`, which can't use a normal index, and malformed JSON can break queries (we hit exactly that: Section 7). Mitigations: filter by indexed columns first (`created_at`, `committed`, `invoice_type`), wrap JSON in a `JSON_VALID` guard, and use indexed generated columns where a field is searched often (customer phone). If I redid it, I'd promote the hot fields (store code, seller, totals) to real columns at write time.

Q. How is authentication done?

Login is username/password with bcrypt, plus Google Sign-In using the server-side Authorization Code flow with PKCE. After login, **the client sends the user id on every request (`x-user-id`)**, and sensitive handlers re-read the role and `is_active` from the DB. **Be honest:** there's no signed session/JWT, so identity isn't cryptographically proven per request. It's acceptable only because it's an internal app on the store LAN. It's documented as the #1 finding in our security review, and the fix is signed session tokens + a role guard on read endpoints before any internet exposure. Interviewers respect "I know the weakness and here's the plan" much more than a bluff.

Q. How do you deploy? CI/CD?

A single Node process on the client's server; the frontend is a Vite build served by it. There's no automated CI pipeline: PRs use a template and are reviewed by two engineers, and I run the test suites locally: ~175 backend test files (Node's built-in `node:test`) and ~110 frontend files (Vitest). Risky features ship **dark** behind env flags and are enabled per environment. DB changes are dated SQL blocks run manually at deploy. *What I'd improve:* CI running tests on every PR, and versioned migrations (the project has Knex but mostly uses hand-run SQL).

Q. How do you know the $8M / 3,000 number?

It's a SQL count over committed SALE tickets (RG + phone orders) in `invoices_archive`, summing `form_json.total`. Same filter the Sales Analytics dashboard uses (Appendix A1). Have the date range ready.

Q. What's the scale? Requests per second?

Be honest: low QPS. It's a few dozen staff across two stores. The hard part isn't throughput, it's **correctness under concurrency** (many terminals pressing buttons on the same queue at the same moment), **money correctness** (rounding, tax, split commissions), and **integrations that fail**. Don't pretend it's high traffic; pivot to the correctness problems, which are real.

Q. What's "demo mode"?

Training environment in the same app: requests with `X-CFC-Env: DEMO` are routed to `demo_`-prefixed tables by a table-name resolver, so trainees can practise without touching real sales. Only admins/managers may enter it; a write guard blocks demo writes into live tables.

4. Rotation queue concurrency (bullet 2)

> **30-second pitch:** "When a customer walks in, the next salesperson in rotation takes them: that's an 'UP'. Every terminal on the floor can press buttons on the same per-store queue at the same time, so the classic risks are lost updates and two people getting the same position. I built a small transactional primitive, `withStoreTx`. It takes a per-store MySQL advisory lock, opens a transaction, locks the queue rows it reads, and retries a bounded number of times on deadlock. The hand-off (Be Back), abandoned-customer and reward flows run inside it in production. I also built a V2 fairness engine on top of it and dark-launched it behind an off/shadow/on flag."

How `withStoreTx` works (backend/src/services/upsFairness.js, 532 lines)

```
for attempt in 1..3:                                  // MAX_TX_RETRIES = 2
   conn = pool.getConnection()
   acq  = SELECT GET_LOCK('ups_queue_lock:' + store, 5)   // 5 s timeout
   if acq != 1: throw LOCK_TIMEOUT  -> HTTP 503 {retryable:true}
   try:
      BEGIN
      rows = SELECT ... FROM ups_queue ... FOR UPDATE      // row locks
      ...apply change, write ups_logs...
      COMMIT
      return result
   catch err:
      ROLLBACK
      if err.code in (ER_LOCK_DEADLOCK, ER_LOCK_WAIT_TIMEOUT) and attempt <= 2: continue
      throw DB_ERROR (500)
   finally:
      SELECT RELEASE_LOCK(name); conn.release()
```

Why two kinds of lock?

- **Advisory lock (`GET_LOCK`)** = a named mutex held by the DB connection. One name per store, so Arden and Waynesville never block each other, and all queue writers in a store are serialized. It also protects operations that *insert* new rows or re-number positions, where row locks alone leave gaps (phantoms).
- **`SELECT … FOR UPDATE`** = an InnoDB exclusive row lock inside the transaction. It guarantees the rows you just read can't be changed by another transaction before you commit (no lost update), even from a code path that doesn't take the advisory lock.
- Together: coarse, cheap serialization per store + precise row-level correctness. Contention is tiny (a handful of clicks per minute per store), so serializing per store costs nothing.

Shadow rollout: say it precisely

Env flag `UPS_FAIRNESS_V2 = off | shadow | on`, default off. In **shadow**, V1 stays the source of truth and V2's session bookkeeping (`ups_sessions`) is **dual-written**, with errors swallowed so V2 can never break V1. **On** switches the take/finish/insert paths to the transactional V2. Production today = **off (V1)**.

> **Things you must NOT say:** "the live rotation is fully transactional", "we compared V1 and V2 in shadow", "we rolled it out store by store". None of that is true in code: there's no diff logging, and the flag is process-wide, not per store. What you *can* say: "I built V2 and the dual-write so we could switch safely; the business hasn't prioritised flipping it, so production still runs V1, while the flows most exposed to races (hand-offs and rewards) already run inside the locked transaction."

Drill-down questions

Q. What race were you actually protecting against?

Example: a Be Back. A manager hands a returning customer to salesperson A. At the same moment A's own terminal presses "take UP", or another manager hands the same customer to B. Without locking, both read "A is at position 3, count 5", both write, one update is lost, and counts and positions drift. With the store lock + FOR UPDATE, the second request waits, then reads the already-updated rows.

Q. Why not just rely on the transaction isolation level?

InnoDB's default is REPEATABLE READ. A plain `SELECT` inside it is a *consistent snapshot* read, so two transactions can both read the same old value and both write: a lost update. You need a **locking read** (`FOR UPDATE`) or an atomic conditional `UPDATE`. SERIALIZABLE would work too but turns every read into a shared lock and causes more deadlocks.

Q. Why do you still get deadlocks if everything is under one store lock?

Not every code path takes the advisory lock. The V1 paths and daily jobs do plain writes, and InnoDB gap/next-key locks from a range `FOR UPDATE` can still conflict with them. Deadlocks are normal in InnoDB: it detects them and kills one transaction with error 1213 (`ER_LOCK_DEADLOCK`). The correct reaction is to roll back and retry the whole transaction. That's why the retry exists, bounded at 2 so a real bug can't loop forever.

Q. Your retry has no backoff. Is that a problem?

Honest answer: it retries immediately on a fresh connection. At our contention level, one immediate retry almost always succeeds because the other transaction has already committed. At higher contention I'd add small randomized backoff (e.g. 20–100 ms jitter) to avoid the two transactions colliding again in lock-step.

Q. What if the lock isn't acquired in 5 s?

It returns HTTP 503 with `retryable: true` instead of hanging the request. It's never happened in practice at our load; 5 s is far longer than any queue transaction (milliseconds).

Q. What if the Node process crashes while holding GET_LOCK?

A MySQL advisory lock belongs to the **connection**. When the connection dies, MySQL releases the lock automatically, so there's no stuck lock. The `finally` block releases it on the normal and error paths. One subtlety: you must run GET_LOCK, the transaction and RELEASE_LOCK on the *same* pooled connection, which the code does.

Q. Advisory lock vs Redis lock?

We have one DB and one app server, so the DB lock is free, needs no new infrastructure, and dies with the connection. With several app servers and no shared DB you'd use a Redis lock with a TTL and fencing token. Here that would be extra infrastructure for no gain.

Q. How do you handle double-clicks?

Idempotency at the endpoint: `/join` returns early if you're already queued today; a repeated `/take` on a closed floor returns `duplicate: true` instead of acting twice; Self Be Back carries an `idempotency_key` with a UNIQUE index; rewards have UNIQUE(user, store, day, kind), so a second insert fails harmlessly.

Q. Why hasn't V2 been turned on?

It changes how rotation order is computed, which directly affects salespeople's commission opportunities. The business wanted the existing behaviour stable while fixing reporting parity bugs first (board display vs rotation tally). The dual-write keeps V2's data warm so the switch is a config change. My next step would be real shadow *comparison*: compute V2's decision alongside V1, log mismatches, and flip once mismatches are zero for a few weeks.

Q. Tell me about a bug in this area.

The "lunch-return" incident (Section 10, story C): a failure in a reward insert inside the same transaction rolled back the salesperson's return to the queue. Lesson: **transaction boundaries must match business atomicity**. Side-effects that are "nice to have" (rewards, audit) shouldn't be able to roll back the core action.

5. Retry-safe sale capture (bullet 3) — your strongest story

> **30-second pitch:** "A sale can only be saved if the salesperson actually served a customer on the rotation board. Finishing with a customer mints a **single-use sale token**, and saving the ticket consumes it with one atomic conditional UPDATE. The ticket itself goes from draft to committed with a **compare-and-set**: `UPDATE … WHERE slug = ? AND committed = 0`, and we check that exactly one row changed. So a double-click, a network retry or a second browser tab can never create a second sale or overwrite a finished one. I also did the RCAs when this path failed in production and shipped the fixes."

The flow

```
1. POST /api/ups/finish        -> opens ups_finish_logs row (salesperson finished with a customer)
2. POST /api/ups/sale-token    -> requires open finish log + customer_acquisition row
                                  INSERT sale_authorization_tokens(token=32 random bytes hex,
                                  user_id, store_name, finish_log_id, expires_at = now+720min)
                                  UNIQUE(finish_log_id)  -> one token per served customer
3. Salesperson builds ticket; drafts autosave:
       upsert ... IF(committed=0 AND form_revision=?, new, old)   -- a stale tab can't clobber
4. Print / e-sign              -> UPDATE ... SET live_at=NOW() WHERE slug=? AND committed=0 AND
live_at IS NULL
5. POST /api/invoice/save (commit):
   a. consume token:
      UPDATE sale_authorization_tokens SET used=1, used_at=NOW(), consumed_slug=?
       WHERE token=? AND user_id=? AND store_name=? AND used=0 AND expires_at > NOW()
      affectedRows == 1 ? ok : (consumed_slug == this slug ? idempotent success : 403)
   b. commit ticket (compare-and-set):
      UPDATE invoices_archive SET committed=1, editable=0, form_revision=form_revision+1
       WHERE slug=? AND committed=0          -> affectedRows must be 1
      (no row -> INSERT; duplicate-key -> someone else won -> treat as already committed)
   c. if a later step fails -> releaseSaleConsumption(): used=0 so the user can retry
```

Key ideas to name

- **Compare-and-set (CAS)** in SQL = conditional UPDATE + check `affectedRows`. The DB row lock makes check-and-write atomic, so no separate SELECT is needed.
- **Idempotency key** = the ticket `slug`. It's client-generated per form session and UNIQUE in the DB. A retry of the same save hits the same slug and gets "already done" instead of a new row.
- **Single-use token** ties a sale to a real served customer (business rule), and `used=0` in the WHERE clause makes consumption race-free.
- **Compensation**: if the DB write after consuming the token fails, the token is released. That's a mini saga: you can't hold one DB transaction across the whole request because the PDF render and Drive upload are outside the DB.

Drill-down questions

Q. Two tabs press Save at the exact same millisecond. Walk me through it.

Same slug: both run the token UPDATE. InnoDB row-locks the token row, so one gets affectedRows=1, and the other waits, re-evaluates `used=0` (now false) and gets 0 rows. It then sees `consumed_slug` equals its own slug and returns idempotent success without writing. Different slugs (two different tickets for one customer): the second gets 403 "already used". One customer served = one sale.

Q. Why not a SELECT then UPDATE?

Classic TOCTOU race (time of check to time of use): both requests SELECT "unused", both UPDATE. The conditional UPDATE puts the check and the write in one atomic statement.

Q. What happens if the response is lost after a successful commit?

The client retries with the same slug. The token step sees it was consumed by this slug, so it's an idempotent success. The commit UPDATE matches 0 rows because the ticket is already committed, and the server returns success and re-seeds its cache from the DB row. The user sees success; no duplicate.

Q. Is the invoice immutable after signing?

Answer precisely: once committed, a draft save or a second commit can't overwrite it: the WHERE `committed=0` guards enforce that. We built a "Sale Lock" that freezes fields at print time, but the business put it on hold, so it's disabled. There are deliberate, audited ways to change a committed sale: a manager-approved delete request, and admin back-dating of the date (≤ 7 days, reason required). That's why the resume says "can't be duplicated or overwritten", not "immutable".

Q. Why a 12-hour TTL? Isn't that long for a "token"?

It isn't a security credential; it's a business-rule ticket tied to one served customer and one user/store. It started at 30 minutes, and that caused a real incident: salespeople spend hours on big furniture orders, and the token expired mid-paperwork (Section 10, story A). We moved to 720 minutes plus a 16-hour hard ceiling for recovery.

Q. What's `form_revision` for?

Optimistic concurrency on drafts. Each save sends the revision it was based on; the write only applies if it still matches, otherwise 409. That stopped an old tab from clobbering a newer save (Section 10, story E).

Q. Why not wrap everything in one DB transaction?

The request also renders a PDF (seconds, in Chromium) and uploads to Google Drive. Holding row locks across that is terrible for concurrency, and you can't roll back an external upload anyway. So: short DB transactions + idempotent steps + a compensating release of the token. The Drive upload runs after the response, fire-and-forget, with its own retry and a `gdrive_file_id` column for idempotency.

6. Integrations and report pipeline (bullet 4)

> **30-second pitch:** "The platform talks to about ten external systems: Google Drive, Gmail and Google Sign-In, USPS address validation, DispatchTrack for deliveries, the RETAILvantage ERP by REST and SQL Server, Birdeye for reviews. The main lesson: every third party fails eventually. So every integration has a timeout, retry with exponential backoff, idempotent writes and an env kill switch, and a failure in one never blocks a sale. I also built the daily report pipeline: SQL aggregation → headless Chrome renders a PDF → upload to Drive → email, with retries, per-date idempotency and audit tables."

Integration list (be able to recite)

Service	What for	How
Google Drive	Archive invoice PDFs and reports	googleapis, OAuth refresh token; updates an existing file instead of creating duplicates
Gmail SMTP	Reports, notifications	nodemailer
Google Sign-In	Staff login	Server-side Authorization Code + PKCE; kill switch `GOOGLE_LOGIN_ENABLED`
USPS Addresses v3	Validate delivery address	OAuth; note: HTTP 200 ≠ deliverable, you must check `DPVConfirmation`
DispatchTrack	Delivery analytics / payments	Polled XML export API, fast-xml-parser, idempotent upsert
RETAILvantage (REST + SQL Server)	ERP: sale-number lookup, items, customers, order push	mssql driver (read-only) + REST outbox (push is dark)
Legacy store SQL Servers (2)	Historical UPS data	Backfill tool (bullet 6)
Birdeye	Google-review requests, competitor review sync	REST via axios
Topaz SigWeb	E-signature pads	Browser SDK

Daily report pipeline

```
node-cron '0 19,21 * * *' (America/New_York), isRunning guard
  -> orchestrator: report_job_history (idempotent per business_date unless forced)
  -> SQL aggregation (dailySalespersonSummary.service.js) : sales, UPS taken, closing ratio, splits
  -> HTML template -> Puppeteer: fresh headless Chromium, setContent, wait fonts, A4 landscape PDF
  -> Drive upload (best-effort; failure logged, doesn't block email)
  -> email to REPORT_RECIPIENTS + active admins (deduped); per-recipient failure isolated
  -> up to 3 attempts, 30 s apart; non-retriable errors (no recipients) stop early
  -> audit rows: report_job_history, report_delivery_logs, report_drive_upload_logs
```

> **Know this before the interview:** the scheduled run has been **paused in code since 17 Jun 2026**, after the SMTP server refused port 587 (ECONNREFUSED) and the Drive folder env was missing. Admins trigger it manually from the admin page. If asked "is it running?", say exactly that, then turn it into a lesson: "the audit tables showed the failure immediately, and I disabled the cron rather than let it spam retries until infra fixed SMTP".

DispatchTrack sync

- No webhook: polls `POST /orders/api/export.xml` per account and date. Nightly cron at 9:45 PM for today + yesterday, plus a backfill every 20 minutes walking a 2-year window.
- Progress tracked in `dispatchtrack_sync_log` UNIQUE(sync_date, endpoint, account, env); the backfill picks dates not yet synced (no cursor needed).
- `INSERT … ON DUPLICATE KEY UPDATE` on UNIQUE(dt_order_number, dt_account, env), so re-syncing a day is harmless.
- withRetry: 800 ms base × 2^n, max 8 s, 3 retries; transient failures marked "retry" for the next tick; requests spaced out to respect rate limits; each account isolated.
- "Unsynced" is shown as "incomplete coverage", never as zero. We never show $0 when we just don't have the data.

Drill-down questions

Q. Why exponential backoff with jitter?

Fixed-interval retries hammer a struggling service and synchronize clients (thundering herd). Doubling the delay gives it room to recover; random jitter spreads the retries out. Always cap the delay and the attempt count, and only retry *transient* errors (timeouts, 5xx, connection reset), never 4xx validation errors.

Q. Why poll instead of webhooks?

DispatchTrack's export API is what the client's account supports. Polling also gives natural backfill and self-healing: if a run fails, the next tick re-requests the date. Webhooks would need a public endpoint (this server is on the store network) plus reconciliation anyway, because webhooks get dropped.

Q. How does a PDF render break production? (great story)

The invoice page loaded a third-party signature script (`sigplusweb.com`). When that host stalled, Chromium never reached DOMContentLoaded, the render hit a 20 s timeout, and the Drive archive was skipped, leaving `gdrive_file_id` NULL. Fix: block every non-same-origin request during server-side renders, plus a regression test (Section 10, story D).

Q. What's a kill switch vs a feature flag?

A feature flag gates a new feature (dark launch, gradual enablement). A kill switch turns off something already live when it misbehaves, without a deploy. We have 27 backend + 16 frontend `*_ENABLED` env flags. Most are dark-launch gates; a few are integration kill switches (DispatchTrack, RV push, Google login, OCR, sale lookup). They're env vars read at request time, so flipping one takes a restart, not a code change.

Q. Why launch a new Chromium per report?

Simplicity and isolation. A crash or memory leak can't affect the next render, and the report runs twice a day. For invoices (on the user's critical path) cold Chromium start is the main latency, and a browser pool is the known next optimisation. We already moved the Drive upload off the response path so the user isn't waiting on it.

7. Analytics production-500 RCA (bullet 5)

> **30-second pitch:** "Managers reported that every store showed the same sales numbers. The store filter was built with `JSON_TABLE` inside a correlated `EXISTS`. The production MySQL build rejected that, so every store-filtered request returned 500, and the frontend silently kept showing the 'All Stores' result. I replaced it with a scalar predicate, `JSON_EXTRACT(form_json, '$.sellerNames[0].storeCode') = ?`, wrapped in a `JSON_VALID` guard. I applied it to about 13 analytics endpoints, added regression tests, and wrote a read-only script to validate the store totals against production."

Before / after

```
-- BEFORE (threw on prod MySQL build -> 500)
AND EXISTS (SELECT 1 FROM JSON_TABLE(JSON_EXTRACT(form_json,'$.sellerNames'),
              '$[*]' COLUMNS(storeCode VARCHAR(10) PATH '$.storeCode')) jt
            WHERE jt.storeCode = ?)

-- AFTER
SAFE_FORM_JSON = CASE WHEN JSON_VALID(form_json) THEN form_json END
AND JSON_UNQUOTE(JSON_EXTRACT(SAFE_FORM_JSON, '$.sellerNames[0].storeCode'))
      COLLATE utf8mb4_unicode_ci = ?
```

Follow-up bug, the next day: some old rows had `form_json = ''` (empty string). `JSON_EXTRACT('')` raises "document is empty" and 500s the whole query. Hence the `JSON_VALID` guard: invalid JSON → NULL → the row simply doesn't match.

Drill-down questions

Q. Why did it work locally but not in production?

Different MySQL builds/versions. `JSON_TABLE` exists from MySQL 8.0.4, and correlated references inside it are restricted on some builds. Local was newer. Lesson: run local/staging on the **same DB version** as production. The code now also probes `supportsJsonTable()` before using it.

Q. Why did users see wrong numbers instead of an error?

The frontend kept the previous (All Stores) payload when a store request failed. So a 500 turned into a *silent wrong answer*, which is worse than an error. We fixed the query; the broader lesson is to show a visible error state instead of stale data.

Q. Why `sellerNames[0]`, the first seller only?

Business rule: a sale is attributed to the primary seller's store. Split sales credit the dollars to several sellers, but **counts** go to the primary only, and the store follows the primary seller.

Q. Is that predicate indexed? How does it perform?

Be precise: no. `JSON_EXTRACT` is evaluated per row. MySQL first narrows rows with the indexed `created_at` date range, so it only scans one period's tickets (hundreds to low thousands), which is fast. If it ever became slow, add a **generated column** `store_code AS (JSON_UNQUOTE(JSON_EXTRACT(form_json,'$.sellerNames[0].storeCode'))) STORED` with an index, or write the store code to a real column at save time. *Don't* call this a performance fix: it was a correctness/availability fix.

Q. How did you make sure the fix was right in production?

Regression tests for the filter builder (including empty `form_json`), and a read-only production script that counts malformed JSON rows and checks that Arden + Waynesville + Web = All Stores.

8. SQL Server → MySQL backfill (bullet 6)

> **30-second pitch:** "Each store used to run an old in-store 'UPS management' app on its own SQL Server. To keep that history, I built a backfill tool that copies customers, up-sheets and transactions into MySQL. It reads in keyset pages of 1,000 (`WHERE Id > @lastId ORDER BY Id`), saves the last id after each page, and can resume from that checkpoint after a crash. It retries transient errors with exponential backoff and jitter, upserts on (store, source id) so re-runs are safe, and reconciles source vs target counts at the end."

```
services/legacyMigration/   (~1,600 lines, 14 files)
  extract/mssqlExtractor.js   SELECT TOP (1000) ... WHERE [Id] > @afterId ORDER BY [Id]
  load/repository.js          INSERT ... ON DUPLICATE KEY UPDATE  (UNIQUE source_store, source_id)
                              checkpoint: UPDATE migration_runs SET last_source_id = GREATEST(...)
  runner/migrationRunner.js   resume = start from MAX(last_source_id); --no-resume to restart
  utils/retry.js              delay = min(15s, 500ms * 2^(n-1)) + jitter; retries = 5; transient codes
only
  errors -> migration_errors ; validate -> count reconciliation ; rollback script ; 3-year retention
cutoff
```

Drill-down questions

Q. Why keyset pagination instead of OFFSET?

`OFFSET n` makes the DB read and throw away n rows every page, so it gets slower page by page ($O(n^2)$ overall). And if rows are inserted or deleted mid-run, pages shift and you skip or duplicate rows. Keyset (`Id > last`) uses the primary-key index to jump straight to the next page in O(log n), and the last id is a natural resume checkpoint.

Q. What if it crashes after writing a page but before saving the checkpoint?

On resume it re-reads that page. Because the load is an upsert keyed on (store, source_id), re-writing those rows is harmless. That's the point: checkpoint + idempotent writes = **at-least-once processing with exactly-once effect**.

Q. Which errors do you retry?

Only transient ones: ETIMEOUT, ECONNRESET, ER_LOCK_DEADLOCK and similar. A bad row (data error) is written to `migration_errors` and the run continues. You don't retry something that will fail the same way every time.

Q. How many rows did you migrate?

Only give a number if you personally ran it and saw the log. The repo has no run logs. Safe answer: "It was built and validated as an operator-run tool; the live legacy analytics still read SQL Server directly, so the backfill is the safety copy for when those servers are retired."

Q. Where is the resume logic's weak spot?

(Shows depth.) It resumes from `MAX(last_source_id)` across all runs, including failed ones. That's fine because the checkpoint only advances after a page is written, but a cleaner design would resume from the last *successful* run's checkpoint only, which is what the function name promises.

9. Concept primer (answer these cold)

Concept	Crisp answer
ACID	Atomic (all or nothing), Consistent (constraints hold), Isolated (concurrent transactions don't see each other's partial work), Durable (committed = survives a crash; InnoDB redo log).
Isolation levels	READ UNCOMMITTED < READ COMMITTED < REPEATABLE READ (InnoDB default: snapshot per transaction, next-key locks stop phantoms for locking reads) < SERIALIZABLE. Anomalies: dirty read, non-repeatable read, phantom, lost update.
Lost update	Two transactions read the same value and both write back; one write disappears. Fix: `SELECT … FOR UPDATE`, atomic `UPDATE x = x + 1`, or optimistic version check.
Pessimistic vs optimistic locking	Pessimistic: lock first (FOR UPDATE, GET_LOCK); good under contention. Optimistic: no lock, detect a conflict at write time (`WHERE version = ?`, then affectedRows 0 means retry or 409); good when conflicts are rare. You used both: FOR UPDATE on the queue, `form_revision` on drafts.
Deadlock	Circular wait: T1 holds A and wants B, T2 holds B and wants A. InnoDB detects it and rolls back one (errno 1213). Prevent with consistent lock ordering, short transactions and fewer locked rows; handle it by retrying the whole transaction.
Advisory lock	An application-named mutex managed by the DB (`GET_LOCK(name, timeout)`), owned by the connection, auto-released on disconnect. Doesn't lock data; it serializes code paths that agree to use it.
Compare-and-set	Update only if the current value equals the expected value, atomically. In SQL: `UPDATE … WHERE state = 'expected'` + check affected rows.
Idempotency	Doing it twice = doing it once. Tools: idempotency key + UNIQUE constraint, upsert, conditional update, "already done" returns success. Needed because networks retry.
At-least-once vs exactly-once	Delivery is at-least-once in practice. You get exactly-once *effect* by making the consumer idempotent.
Keyset pagination	`WHERE id > :last ORDER BY id LIMIT n`: constant cost per page, stable under inserts, resumable.
Backoff + jitter	delay = min(cap, base × $2^{attempt}$) + random. Retry transient errors only; bound the attempts.
Dark launch / feature flag	Ship code to production turned off; enable by config; roll back without a deploy. Shadow = run the new path alongside the old without affecting results.
Saga / compensation	A multi-step process across systems that can't share one transaction; each step has an undo (e.g. release a consumed token if the commit fails).
Index on JSON	MySQL can't index a JSON path directly. Use a generated (virtual/stored) column on the extracted value, then index it (or a functional index in 8.0.13+).

10. Incident stories (STAR) for behavioural rounds

Don't name customers or the client in interviews; say "a customer's sale" and "a US furniture retailer".

A. Sale tokens expiring mid-order (Jun 2026)

S: Two completed sales (~$490 and ~$470) could not be saved; the salespeople were blocked at checkout. **T:** Find out why a valid sale was rejected, and make sure it can't recur. **A:** Traced it to the token: its 30-minute TTL expired while salespeople spent hours on large furniture orders. The "re-mint" path then did a blind INSERT, hit the UNIQUE(finish_log_id) key and returned null, which left the user stuck. I raised the TTL to 12 h with a 16 h hard ceiling for recovery, and changed re-mint to an idempotent in-place UPDATE (`WHERE finish_log_id=? AND used=0`). I also added a frontend retry that re-mints once and retries with the same slug. **R:** [Say what actually happened afterwards, e.g. "no repeat since", only if true.] **Lesson:** timeouts must match how the business actually works, and "retry" paths need the same idempotency as the happy path.

B. Every store showing identical analytics (Jun 2026)

See Section 7. **Lesson:** keep production and dev DB versions identical, and never let a failed request silently show stale data.

C. Salesperson vanished from the queue after lunch (May 2026)

S: A salesperson pressed "return from break" and got a 500; they were not back in the rotation. **T:** Put them back and find the root cause. **A:** The return inserted the queue row *and* an optional reward row in the same transaction. The reward insert failed (a new action type was longer than the ENUM allowed: errno 1265 data truncated), which rolled back the queue insert too. Fix: `action_type` ENUM → VARCHAR(64) with an allow-list in code (so new types need no DDL), and the reward moved to a separate transaction so it can never roll back the core action. **Lesson:** transaction scope = business atomicity; side-effects stay outside it.

D. Invoice PDFs not archived to Drive (Aug 2026)

S: Some invoices had no Drive copy (`gdrive_file_id` NULL). **A:** The server-side Chromium render loaded a third-party signature script. When that host stalled, the page never finished loading, the render timed out and the archive step never ran. Fix: block all non-same-origin requests during server renders, plus a regression test (`pdfGenerator.externalBlock.test.js`). **Lesson:** a server-side render must never depend on a third-party network call.

E. Completed sale overwritten by an old draft (Sep 2026)

S: A committed sale's data got replaced by an older draft. **A:** A stale tab's autosave upsert overwrote the committed row. Separately, an idempotent replay reported success without writing, and the PDF was rendered from an in-memory cache written *before* the DB accepted the save. Fix: a guarded draft write (`IF(committed=0 AND form_revision=?, new, old)`; 0 rows → 409), a `form_revision` column, and the cache now filled *from* the DB row after the write. **Lesson:** the DB is the single source of truth; caches are written from it, never ahead of it.

Questions to expect about you as the sole engineer

Q. How do you avoid breaking production with no QA team?

Dark launch behind flags, a regression test for every incident (node:test backend, Vitest frontend) (a test named after the bug), read-only production validation scripts before and after risky changes, two-reviewer PRs, and dated SQL change blocks run at deploy.

Q. Biggest mistake?

Story A (30-minute TTL): I designed the token around the "happy path" timing, not real salesperson behaviour. Now I check timeouts against production data before choosing them.

Q. What would you rebuild?

(1) Signed sessions instead of trusting a client-sent user id. (2) Promote hot JSON fields to real indexed columns. (3) CI running tests on every PR + versioned migrations. (4) Flip rotation V2 via real shadow comparison.

Appendix A. Read-only SQL to verify the numbers

Run these yourself on the `cfc_form` database (SELECT only). Write the results and the date you ran them next to each resume number, so that in the interview you can say "as of <date>".

A1. Sales tickets and written sales (same filter as the Sales Analytics dashboard)

```
SELECT COUNT(*) AS tickets,
       ROUND(SUM(CAST(JSON_UNQUOTE(JSON_EXTRACT(form_json,'$.total')) AS DECIMAL(12,2))),2) AS
written_sales,
       MIN(created_at) AS first_ticket, MAX(created_at) AS last_ticket
FROM invoices_archive
WHERE committed = 1 AND transaction_mode = 'SALE' AND invoice_type IN ('RG','PH')
  AND JSON_VALID(form_json);
```

A2. Split by store (confirms the "2 showrooms + web" wording)

```
SELECT JSON_UNQUOTE(JSON_EXTRACT(form_json,'$.sellerNames[0].storeCode')) AS store,
       COUNT(*) AS tickets,
       ROUND(SUM(CAST(JSON_UNQUOTE(JSON_EXTRACT(form_json,'$.total')) AS DECIMAL(12,2))),2) AS sales
FROM invoices_archive
WHERE committed = 1 AND transaction_mode = 'SALE' AND invoice_type IN ('RG','PH') AND
JSON_VALID(form_json)
GROUP BY store;
```

A3. Queue activity (only if you keep a queue number)

```
SELECT COUNT(*) AS queue_events,
       COUNT(DISTINCT DATE(created_at)) AS trading_days,
       MIN(created_at), MAX(created_at)
FROM ups_logs;
-- optional: by action type
SELECT action_type, COUNT(*) FROM ups_logs GROUP BY action_type ORDER BY 2 DESC;
```

A4. DispatchTrack records

```
SELECT COUNT(*) AS orders, COUNT(DISTINCT stop_key) AS stops,
       MIN(delivery_date), MAX(delivery_date)
FROM dispatchtrack_orders WHERE cfc_env = 'LIVE';
```

Column names in A3/A4 come from `schema.sql`. If one errors, run `DESCRIBE ups_logs;` / `DESCRIBE dispatchtrack_orders;` and adjust.

Appendix B. Code map (open these the night before an interview)

Topic	Files
Rotation locks	`backend/src/services/upsFairness.js` (withStoreTx, GET_LOCK, retries, flag); `backend/src/routes/ups.routes.js`; `beBack.service.js`, `abandonedUp.service.js`, `selfBeBack.service.js`
Sale token / CAS	`backend/src/services/saleToken.service.js`; `backend/src/routes/route.invpdf.js` (save/commit); `invoicePersistence.service.js` (persistDraftForm); `docs/SALE_TOKEN_ROLLOUT.md`

Reports	`backend/src/jobs/dailySalespersonSummary.job.js`; `services/reports/dailySalespersonSummary.*.js` (service, pdf, drive, orchestrator, audit, recipients)
DispatchTrack	`backend/src/services/dispatchtrack/` (dtClient, dtWorker, dtSync); `jobs/dispatchtrack.job.js`
Analytics store filter	`backend/src/routes/admin/analytics.legacy.routes.js` (lines ~136–215); `backend/tests/analytics.storeFilter.test.js`; `backend/scripts/prod-validate-store-filter.mjs`
Legacy backfill	`backend/src/services/legacyMigration/` (extract, load, runner, utils/retry.js, sql/001_legacy_schema.sql)
Security model	`documentation/SECURITY_REVIEW.md`
PDF render stall	`docs/features/pdf-generation.md`; `backend/tests/pdfGenerator.externalBlock.test.js`